

Duckietown – Vision-Guided Navigation and Live Chat

Tair Kezdekbayev Alvaro Mendez Li Tan Kayra Erol

*Department of Computer Engineering
Technical University of Munich
Heilbronn, Germany*

1 ABSTRACT

This project implements autonomous right-lane navigation for a Duckiebot using ROS 1, monocular camera perception, a directed road map, and a desktop companion application. The onboard controller segments yellow, white, and red road markings, estimates lane position, regulates differential wheel commands, and switches to bounded junction manoeuvres when lane borders disappear. A directed A* planner calculates the shortest legal route between red-line approaches. The Windows application combines route selection, live camera diagnostics, status reporting, a working local live chatbot, and a direct stop control. The software was matched to the robot’s installed ROS Noetic environment and evaluated with native tests, network-isolated ROS integration, stationary live checks, wheel diagnostics, and supervised track trials. Straight driving, a left curve-to-straight transition, a sharp right road bend, and individual straight, left, and right junction traversals were demonstrated. The chatbot provides immediate timed pauses, straight-road speed profiles and validated junction instructions. Its command rules run locally without model training or a cloud service.

2 INTRODUCTION

Duckietown provides a compact autonomous-driving platform in which perception, control, distributed software, and physical testing can be studied together [1]. The objective of this project was to make one robot, *duck2*, follow the right lane and reach a selected red-line destination on a known course. The solution had to operate with the camera and wheel interfaces already installed on the robot, tolerate missing borders inside intersections, and expose an understandable interface to the operator.

The resulting system divides responsibility between the robot and laptop. The robot performs all time-critical perception, steering, wheel output, and safety checks. The laptop selects a directed start, computes a route, sends validated high-level commands, maintains a live-chat session, and visualises camera and controller data. Communication uses an authenticated SSH tunnel, so the HTTP command and camera services remain bound to loopback on the robot.

3 SYSTEM ARCHITECTURE

The robot host runs Ubuntu 18.04.6 on ARM64. Its installed ROS container provides ROS Noetic, Python 3.8.10, OpenCV 4.2.0, and NumPy 1.17.4. Development images use the digest-pinned `duckietown/dt-ros-commons:v4.3.0` base for both AMD64 and ARM64. The camera input is `/duck2/camera_node/image/compressed` with `sensor_msgs/CompressedImage`; wheel requests use `/duck2/wheels_driver_node/wheels_cmd` with `duckietown_msgs/WheelsCmdStamped`.

The project consists of four main parts:

1. a ROS camera and lane-processing node;
2. a navigation state machine for road segments and junctions;
3. a command gateway, camera gateway, and independent watchdog; and
4. a Windows companion application for map planning and supervision.

Only the lane follower publishes wheel requests during project operation. The gateway accepts structured high-level commands, while the companion subscribes to status and camera data without becoming a second wheel controller.

4 LANE PERCEPTION

4.1 Colour segmentation

Each compressed frame is decoded in OpenCV, converted from BGR to HSV, and cropped to a lower road region. The deployed colour intervals are

Marking	Lower HSV	Upper HSV
Yellow	(20, 70, 80)	(35, 255, 255)
White	(0, 0, 150)	(180, 55, 255)

Connected components and row-wise boundary samples remove small fragments and separate longitudinal lane edges from transverse markings. The controller uses ordered yellow-left and white-right evidence whenever both are available. A short temporal width estimate supports partial observations; fresh frames with insufficient road evidence still remain distinct from missing camera data.

4.2 Lane geometry

Let $x_y(r)$ and $x_w(r)$ be the detected yellow and white boundary positions at image row r , and let W be image width. The normalised lane centre is

$$c(r) = \frac{x_y(r) + x_w(r)}{2W}, \quad e(r) = c(r) - c_{\text{ref}}. \quad (1)$$

Matched image rows in the junction corridor estimator prevent perspective from mixing unrelated depths. The change in $c(r)$ between near and far rows gives an image-based heading estimate, while the near value gives lateral displacement. The ordinary lane reference is $c_{\text{ref}} = 0.441$; the dedicated straight-junction approach uses a separately tuned reference of 0.49. These image references are controller parameters rather than metric lane coordinates.

4.3 Steering control

Lane error is low-pass filtered,

$$\hat{e}_t = \alpha e_t + (1 - \alpha) \hat{e}_{t-1}, \quad \alpha = 0.20, \quad (2)$$

then mapped to bounded steering with gain scheduling, a deadband, and slew-rate limiting. For centre speed v and steering request δ , the wheel command pair is

$$u_L = v - \delta, \quad u_R = v + \delta. \quad (3)$$

The continuous launcher uses $v = 0.09$, a wheel cap of 0.20, a steering cap of 0.11, and proportional gains of 0.35 near centre and 0.75 farther away. These values are normalised driver requests rather than metric velocities. The installed driver adds a minimum PWM offset; consequently, small numerical wheel differences do not always produce equally visible changes in motion.

5 ROAD AND JUNCTION NAVIGATION

5.1 Smooth and sharp roads

Smooth curves remain under continuous visual lane control. A confirmed left road curve can request the supported 0.03/0.20 left/right profile; this boost is gated by curve evidence and is not applied to every straight-road frame. A sharp right road bend is recognised only during ordinary following, after recent valid lane evidence and a confirmed disappearance pattern at the right boundary. The bounded corner state first preserves forward progress and then applies the physically verified 0.20/0.00 left/right pivot request. It returns to visual centering after a stable outgoing corridor appears. Corner recognition is disabled while a junction manoeuvre is active, preventing both state machines from controlling the same event.

5.2 Red-line stopping

A red transverse component near the bottom of the camera region latches a stop. The robot outputs zero for a two-second dwell and waits for the route controller to authorise one valid exit. A red line is therefore a navigation event rather than merely a colour cue. Persistent images of the same departure line cannot advance the route twice.

5.3 Unmarked intersections

The course has areas inside intersections where neither longitudinal boundary is visible. These areas are handled as explicit states:

1. *approach*: ordinary visual lane following;
2. *stopped*: latched zero output and route validation;
3. *crossing*: a direction-specific bounded motion profile;
4. *reacquisition*: search for an ordered outgoing corridor; and
5. *following*: visual control resumes after stable alignment.

Missing lane markings are permitted only in the authorised crossing and search states. Straight crossing uses a stronger equal-wheel request with limited encoder balancing. The left profile reproduces the verified gradual curve with an outer-wheel advantage. The right profile uses forward clearance followed by the calibrated sharp pivot. Reacquisition requires near-field support, correct yellow/white order, acceptable lateral and heading errors, and a stability interval. Every manoeuvre has a deadline.

6 ROUTE PLANNING

The supplied course is represented as five junctions, $A-E$, with directed approaches such as $A \rightarrow B$. A state stores the previous and current junction, which fixes the robot's heading and right-lane position. Legal forward transitions are labelled *left*, *straight*, or *right*. Red-line approaches, rather than undirected junction points, are used as selectable destinations.

The laptop applies A^* search [3]. Every crossed junction has unit cost, and a relaxed graph distance is the admissible heuristic. Therefore, the result minimises the number of junctions while respecting directed lanes. Before execution, the route is converted into a sequence of turn instructions. The next instruction is released only for the current run and junction index; route progress advances only after the robot reports the corresponding outgoing lane.

7 COMPANION APPLICATION

The `duck2_companion.py` desktop application combines two work areas. The map view lets the user choose a directed starting lane and destination red line, shows the A^* route, and requires explicit placement confirmation before Start. The camera/status view displays the normal image, colour mask, or diagnostic overlay together with controller phase, next junction, wheel request, and stop reason. A prominent Stop control remains available throughout a run.

The application connects to loopback ports 8765 and 8766 through a strict-key Windows OpenSSH tunnel. The robot-side services also bind to loopback. This keeps command transport separate from ROS network discovery and makes session startup repeatable after laptop or robot reboots.

8 LIVE CHATBOT

8.1 Implementation and command flow

The live chatbot is integrated into the companion's camera/status panel. The normal application enables it by default, without a test-scenario launcher or an assumption that every starting road is straight. The operator types a message and presses Send or Enter; the input is not populated with a scripted demonstration. `live_chat.py` normalises text, recognises an allowed intent and applies it to the current session. The UI reports actual controller acknowledgments and state changes. It does not claim that an accepted wheel request proves physical movement.

The laptop keeps the future turn queue and validates proposed changes against the same directed map used by A^* . A request such as "go straight at the next junction" replaces the remaining queue with one legal straight instruction. It changes the decision at a red-line stop; it does not disable lane steering on subsequent curves. A turn already accepted for a crossing is completed before another queued turn can be dispatched.

At each red stop the robot observes its two-second dwell and accepts one instruction tied to the current run, approach and junction index. Repeated delivery uses the same identifier, preventing a lost acknowledgment from consuming two turns. The laptop updates its reported lane only after matching the outgoing-lane completion report. With no queued instruction, the chatbot asks where to go next and displays a 60-second waiting limit measured from arrival. Invalid messages do not restart that waiting clock.

8.2 Supported commands

The slow, normal and fast profiles request centre speeds of 0.09, 0.10 and 0.11, respectively, with a straight-profile wheel cap of 0.12. Normal is the initial selection. These profiles apply only when stable visual evidence classifies the road as straight; curve and junction profiles keep their separate settings. Values are normalised driver inputs, not mea-

Table 1: Examples of the implemented live command grammar.

Message	Effect
stop for 7s	Pause immediately on acceptance, then resume after seven seconds if healthy.
go straight at the next junction	Replace pending turns with straight.
go left, go right	Choose the next legal exit.
left after that	Append left to the future queue.
clear turns	Empty the queue; ask at the next red stop.
slow speed, normal speed, fast speed	Select a straight-road speed profile.
speed up, slow down	Move one profile higher or lower.
status	Show reported position and pending instructions.
stop, pause	Pause awaiting Continue for at most 30 seconds.
continue, resume	Resume a pause without bypassing a red stop.
quit, end run	End the session and cancel automatic resumption.

sured speeds.

A timed pause requests zero output in the robot’s command callback before acknowledgment, without waiting for another camera frame. Its duration is measured with the robot’s monotonic clock. Network delivery and mechanical braking still introduce latency. The queue survives the pause, while camera, heartbeat and watchdog checks remain active. A fault or Stop ends the run and prevents automatic resumption. Timed requests accept positive durations up to 3,600 seconds; the shorter 30-second limit applies to an untimed pause.

8.3 How the chatbot was developed

The chatbot uses a hand-written, deterministic command grammar. It was developed by specifying intents, writing phrase-recognition rules and checking examples against parser, session, UI and controller tests. It has no trained neural network, fitted weights, fine-tuning dataset or online learning stage. “Training” in this project therefore means refining and verifying the command rules with examples, not machine-learning training.

Text processing tolerates capitalisation, common punctuation and supported polite phrasing. Explicit route state supplies context for “after that”, relative speed changes and the next junction. Unsupported wording and ambiguous requests produce clarification; arbitrary spelling mistakes are not guessed. No API key or downloaded language model is needed. Historical model experiments are separate from this working implementation.

Duckietown supplies the ROS runtime and development scaffolding. Its official template catalogue describes ROS, core and other project templates [5]; the current command interpreter and live-session policy are project application code. The inspected runtime records do not identify a preinstalled chatbot service.

9 SAFETY SUPERVISION

Movement requires all of the following: a fresh camera frame, a healthy controller, sole wheel-publisher ownership, a current laptop heartbeat, valid route state, and explicit user confirmation. The independent watchdog monitors camera and status freshness, wheel feedback, encoders, publisher owner-

ship, and parent-process health. The controller requests zero after a manual Stop, stale data or an expired manoeuvre. On a detected fault, the independent watchdog asserts the installed driver emergency stop and ends the temporary controller process group.

Cleanup waits for temporary publishers to exit and confirms zero wheel feedback. Restoration of normal robot control is a deliberate step after the project session ends; it is not an automatic app reconnection action. This was especially important during supervised tuning, where each physical trial used a bounded session and ended before the next adjustment.

10 VERIFICATION

10.1 Offline and integration checks

Native checks cover Python syntax, documentation links, Windows launchers, directed A* routes, UI behaviour, command recognition, session identity, deduplication, immediate pause/resume and turn delivery. The live-navigation tests exercise the actual controller with mocked ROS, including profile isolation, red-stop waiting and Stop priority. These are software checks rather than evidence of road containment.

The release review completed 562 native tests: 559 passed and three were skipped for platform-specific conditions. Syntax, documentation links and launcher targets passed for 80 Python sources. Network-isolated HTTP/ROS checks verified all three camera views, each speed profile, immediate pause output, and final-red stopping. A requested seven-second pause measured 7.093 seconds between zero and resumed synthetic wheel requests. The normal companion flow also passed map-queue loading, a straight override, the next red-line prompt, a subsequent left instruction, and terminal Stop. These checks used synthetic images and no robot connection; they establish command integration rather than physical route accuracy. The broader isolated ROS suite also passed, covering route progression, red dwell, camera and heartbeat loss, pause/resume, and explicit Stop.

10.2 Robot and track results

Authenticated Windows SSH, ROS master access, message fingerprints, containers, camera decoding, wheel topics, both encoders, IMU, and the front range sensor were verified on duck2. Stationary checks decoded normal, mask and overlay views and sampled timestamp age. Frame spacing and frame age were recorded as distinct quantities.

Table 2 separates physical observation from controller evidence.

The operator also reported the straight-path live-chat three-second pause check successful. Subsequent combined runs exercised the seven-second pause and a straight override at the first red line. A battery-interrupted attempt remains incomplete. In the latest attempt on 10 September 2026, the operator reported insufficient curve following followed by a stop. Consequently, complete-route physical validation remains *incomplete*, despite accepted component trials and verified command delivery.

Implemented chatbot functionality and accepted individual manoeuvres do not establish identical behaviour across a complete route under changing lighting, battery state and starting pose. Physical observations remain separate from controller acknowledgments and encoder evidence.

Table 2: Historical user-accepted supervised results; not a full-route certificate.

Scenario	Result
Straight lane	Near-perfect eight-second run; prompt stop. The detailed recording of one accepted run was lost; its summary was retained.
Left curve to straight	Stayed contained through the curve, corrected on the straight, and stopped centred in the accepted trial.
Sharp right road bend	Visually accepted turn using the bounded pivot; zero feedback and normal ownership restored.
Straight junction	Straight approach, red stop and dwell, crossing, smooth outgoing centring, and no boundary crossing in the accepted trial.
Left junction	User-observed smooth, correctly directed crossing and outgoing-lane entry; accepted without requested changes.
Right junction	Accepted as passable: no boundary crossing, with a visible leftward heading remaining after entry.

11 DISCUSSION

The most significant engineering issue was that image geometry, wheel commands, and physical motion are related but not interchangeable. Earlier controllers averaged boundary points from different image depths, causing apparent centring to include a heading error. Row-matched geometry separated these effects. The minimum-PWM mapping also meant that small command differences produced nearly equal wheel rotation. Stronger, direction-specific profiles were therefore used only where physical trials supported them.

The state-machine design resolved a second limitation: lane loss can indicate either a fault on a marked road or normal geometry inside an intersection. Allowing missing borders globally would weaken containment; stopping on every missing border would prevent crossings. Phase-dependent interpretation preserves normal lane-loss stopping while allowing a selected manoeuvre to continue until the outgoing corridor is established or its deadline expires.

The software remains map-relative rather than globally localised. The operator must select the actual starting approach. Physically relocating the robot requires a new session rather than relying on its previous reported position. The normal application uses the selected map route as its initial queue and permits live overrides at subsequent red stops. Optional scenarios limit a run to a pause, a crossing, or a configured final red line. These limits do not apply to the ordinary companion. The integrated software flow is implemented; the latest curve failure still requires physical follow-up.

12 REPRODUCIBLE OPERATION

Windows OpenSSH is the verified connection path. A protected key is installed once and unlocked when needed; strict host verification is retained as hotspot addresses change. After boot, the connection-check shortcut verifies reachability and runtime ownership. The driving-preparation shortcut stages the current source, establishes exclusive project con-

trol and leaves the controller stopped awaiting a route. Opening the companion then connects the map, camera and chat. Only the operator’s confirmed Start begins the session.

Run `tools/Start-Duck2-DrivingMode.cmd` while stationary, then `laptop/Start-Duck2Companion.cmd`. Select the real directed start and a red-line destination, connect, confirm placement and press Start. The message box is empty; the operator types every pause and direction message. Camera-only mode uses a separate driving-disabled backend. Docker Desktop is required for local ROS builds and isolated integration tests, but not for this source-mounted robot session. Repository startup, usage and live-chat guides document the exact entry points, dependencies, recovery procedure and supported grammar.

13 CONCLUSION

The project integrates camera perception, closed-loop lane steering, red-line stops, unmarked junction traversal, directed A* planning and a working live chatbot on the installed Duckietown ROS platform. The operator can pause for a specified duration, select a straight-road profile and revise future junction choices through validated text commands. Explicit session state and acknowledgments connect those commands to the robot without placing language interpretation in the wheel-control loop. The recorded results establish implemented software and accepted component trials while retaining the incomplete status of the latest combined physical attempt.

REFERENCES

- [1] L. Paull *et al.*, “Duckietown: An open, inexpensive and flexible platform for autonomy education and research,” in *Proc. IEEE Int. Conf. Robotics and Automation*, 2017, pp. 1497–1504.
- [2] M. Quigley *et al.*, “ROS: An open-source Robot Operating System,” in *ICRA Workshop on Open Source Software*, 2009.
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [5] Duckietown, “Project Templates,” *Developer Manual*, accessed 10 September 2026. [docs.duckietown.com, Project Templates](https://docs.duckietown.com/Project%20Templates).
- [6] T. Kezdekbayev, A. Mendez Li, and T. K. Erol, “AlTaTa Duckietown Final,” software repository, 2026. [GitHub: DuckietownTUM/AlTaTa-Duckietown-Final](https://github.com/DuckietownTUM/AlTaTa-Duckietown-Final).